



[BIT703 Web Technologies](#) / [Contents](#) / Topic 10: Server-side web application development with procedural and object-oriented database programming / PHP OOP

Inheritance 30m

Inheritance is an established principle in many OO programming languages. We learned about the concept of inheritance earlier and learned how inheritance affects the level of detail (available attributes and methods) through each inherited class.



If you need a refresher about this, we recommend taking a look at the following pages:

[Cascade, inheritance and specificity →](#)

In PHP OOP, inheritance is the concept of a class deriving from another class. This means a child class will inherit characteristics from its parent class. Inheritance is useful because it means a class can inherit the same functionality as the parent class without needing to write the same code again. We use this concept to augment an existing class with features that the child class does not (and does not need to) contain. Parent classes tend to be classes that contain mostly common attributes and methods that inherited classes can use without reinventing the class. All the inherited class would require would be the things that make it different from the parent.

By using inheritance, you can create reusable blocks of code that you only write once for the parent class, which is then inherited by the child classes.

To declare that a class is to inherit from a parent class, you must use the keyword 'extends'. The syntax for inheritance is as follows (where Child is the child class and Parent is the parent class):

Skip to main content
`class Parent {`

```
// code for Parent class  
  
}  
  
class Child extends Parent {  
  
    //definition body  
  
}
```

The effect of inheritance is that the child class (or subclass or derived class) will:

- automatically inherit all the member variable declarations of the parent class
- automatically inherit all the same methods as the parent, which (by default) will work the same way as those functions do in the parent.

In addition, a child class can also have its own attributes and methods on top of the ones it inherited from the parent class, which can add more functionality to the child class.

It is important to note that a child class can inherit code from the parent class, but the parent class **cannot** inherit code from the child class.

Base class

When a class has no parent class, then this class is referred to as the base class and often just includes attributes that would be common to all the inherited or derived classes. For example, the attribute 'pages' would be common to all types of books and found in the base class Book. The base class is also recognised as the class at the top of the hierarchy of inherited classes.

In the following example, Book is the base class, and Novel is the extended (derived) class. This example shows that Novel inherits the Book class and adds more functionality based on the requirement. Apart from the inherited functions, Novel also adds two additional member functions.

```
class Novel extends Book {
```

[Skip to main content](#)

```
var $publisher;  
  
function setPublisher($par){  
  
    $this->publisher = $par;  
  
}  
  
function getPublisher(){  
  
    echo $this->publisher. " ";  
  
}  
  
}
```

In PHP, **__set()** is used when a value is to be assigned to an undefined variable of a class and **__get()** is used when the value from an undefined variable is to be read.

You will find the complete code for this inheritance below. Have a go at executing this code yourself to understand inheritance.



[Skip to main content](#)

```
1  <?php
2
3  class Book {
4
5      /* Attributes */
6      public $price;
7      public $title;
8      public $author;
9
10     /* Methods */
11     public function getPrice(){
12         echo $this->price.'<br/>';
13     }
14
15     public function getTitle(){
16         echo $this->title.'<br/>';
17     }
18     public function getAuthor(){
19         echo $this->author.'<br/>';
20     }
21
22
23     public function __construct($_title,$_price,$_author){
24         $this->title = $_title;
25         $this->price = $_price;
26         $this->author = $_author;
27     }
```



This code will produce the output:

Computer Book

85

Author A

Yes

[Skip to main content](#)

The Mirror & the Light

Traits

One of the most important aspects of OOP is the ability to reuse code. We've just learnt that inheritance can be used so the attributes and methods declared in the parent class can be inherited by the child classes. Inheritance can be used if the classes are from the same class hierarchy.

An inheritance tree is another name for a class hierarchy. It is a tree-like structure with the parent class at the top of the hierarchy (tree) and all the other child classes descending from the parent class below it.

However, inheritance makes the code very tightly coupled. Therefore, overuse of inheritance will cause the inheritance tree to become very deep (that is, multiple child classes becoming derived from one parent class). Having a deep inheritance tree may cause the code to become complex and very hard to maintain.

To overcome this problem, PHP 5.4 introduced **traits**, which allow you to reuse various methods freely in many different classes that do not need to be in the same class hierarchy. This means that a trait can be used to declare methods that can be used in multiple independent classes across different class hierarchies.

Inheritance allows classes to reuse the code **vertically**, while traits allow classes to reuse the code **horizontally**.

How to use traits

A trait is like a class, but it is only intended to group methods in a fine-grained and consistent way.

PHP does not allow you to create an instance of a trait like you can create an instance of a class. Traits are used to add behaviours to classes, rather than to create objects.

To declare a trait, you must use the **trait** keyword. The syntax for a trait is:

[Skip to main content](#)

```
trait TraitName {  
  
    // code  
  
}
```

Then, to use the declared trait in a class, you must use the **'use'** keyword, such as in the code below:

```
class Example {  
  
    use TraitName;  
  
}
```

The example below shows how to use the `trait` and `use` keywords. This is a simple example that shows the trait `message1` being declared. Then, a class is created called `Welcome`.

```
<?php  
  
trait message1 {  
  
    public function msg1() {  
  
        echo "00P is fun! ";  
  
    }  
  
}  
  
class Welcome {  
  
    use message1;
```

Skip to main content

```
}  
  
$obj = new Welcome();  
  
$obj->msg1();  
  
?>
```

In this example, we declared one trait **message1**. We then created a class **Welcome**. This means that the class will use the trait, and all the methods in the trait will be available to the class.

If other classes need to use the `msg1()` function, simply use the `message1` trait in those classes. This reduces code duplication because there is no need to redeclare the same method repeatedly.

Try it out!



☒ Traits

Non-Assessed

[Skip to main content](#)

In this activity, you will execute the code below.

```
<?php
trait message1 {
    public function msg1() {
        echo "OOP is fun! ";
    }
}

class Welcome {
    use message1;
}

$obj = new Welcome();
$obj->msg1();
?>
```

What do you expect the output to be for this code?

Execute the code. Was the output what you expected it to be?

☐ I have executed the code

Submit



☒ Inheritance: Boot class

Non-Assessed

[Skip to main content](#)

In this activity, you will create the PHP code for the scenario given below.

From the **parent class Footwear**, you must create a **new class called Boot** that is to **inherit** all the attributes and methods of the Footwear class. To do this, you must follow the points below.

- The parent class Footwear has attributes for colour and size.
- The new child class must be called Boot.
- The output should display:
 - This footwear has no fasteners
 - This boot has holes for boot laces

Once you've created the PHP code, submit it in the text box below.

☐ I have completed this task

Submit



☒ Traits: Desired class

Non-Assessed

[Skip to main content](#)

In this activity, you will create the PHP code for the scenario given below.

You must create a class called **LandMammal** that can access the methods from both the **base class**, **Mammal**, and the **trait class** **locomotion** (speed of the mammal). To do this, you must follow the points below.

- Create a base class called **Mammal**, which has a single method.
This method is to display the message 'Good day from main class'.
- Create a trait class called **Locomotion**, which has another method. This method is to display the message 'Welcome to traits'.
- Create a class called **LandMammal** that is able to access the methods of both the base class (**Mammal**) and trait class (**Locomotion**), which is seen in its object \$myobject.

☐ I have completed this task

Submit



Thoughts on this page?

